CAPSTONE PROJECT DESIGN DOCUMENT

TRAFFIC-LIGHT CONTROLLER + MOD-N COUNTER

Project Domain: Digital Logic Design / Verilog HDL

Project Type: Finite State Machine (FSM) and Counter Design

Prepared for: Capstone Project Submission

Project Summary

This project describes a digital traffic-light controller for a simple two-road intersection. The controller uses a finite-state machine to select the active signal and a counter to determine how long each signal remains active. A reusable MOD-N counter module is also provided as a separate design component.

Student Name: _____

Register Number: _____

Institution: _____

1. Abstract

The Traffic-Light Controller + MOD-N Counter is a digital system that manages traffic signals at a two-road intersection. The North-South (NS) and East-West (EW) directions receive green, yellow, and red indications in a fixed sequence. A finite-state machine (FSM) controls the sequence, while a counter provides timing. The design is described in Verilog HDL and can be tested using a digital simulation tool. The project demonstrates sequential logic, state transitions, counters, clock signals, reset operation, and hardware description language concepts.

2. Introduction

Traffic signals help organize the movement of vehicles at road intersections. A controller must ensure that conflicting directions are not given green signals at the same time. In a digital implementation, the sequence can be represented using states. A clock-driven counter measures the duration of each state. This project implements a simplified two-road signal system for learning digital design.

3. Problem Statement

Manual or poorly coordinated signal operation can lead to confusion and inefficient traffic movement. The aim of this educational project is to design a predictable digital controller that changes signal indications in a fixed order and uses a counter to control each phase duration.

4. Aim and Objectives

• To design a two-road traffic-light controller using Verilog HDL.

• To implement a finite-state machine with four operating states.

• To use a counter to measure green and yellow phase durations.

• To design a reusable parameterized MOD-N counter module.

• To verify the signal sequence through simulation.

5. Scope of the Project

The project covers a basic two-direction intersection with fixed timing. It does not include vehicle sensors, pedestrian crossing logic, emergency-vehicle priority, adaptive traffic timing, or certified safety features. These can be considered as future improvements.

6. System Overview

The clock provides regular timing pulses. The controller stores the current state and increments a phase counter on each rising clock edge. When the count reaches the configured limit, the controller moves to the next state and clears the counter. The output logic converts the current state into six signal outputs.

7. Block Diagram

The main functional blocks are shown below:

Clock + Reset
FSM State Register
Phase Timing Counter

Output Decoder
NS Red / Yellow / Green | EW Red / Yellow / Green

The timing counter is part of the main controller. The separate MOD-N counter module is included as a reusable counter example for the capstone.

8. Operating Sequence

State	North-South	East-West	Next state
NS_GO	Green	Red	NS_WARN
NS_WARN	Yellow	Red	EW_GO
EW_GO	Red	Green	EW_WARN
EW_WARN	Red	Yellow	NS_GO

9. State Machine Description

The FSM contains four states: NS_GO, NS_WARN, EW_GO, and EW_WARN. After reset, the controller starts in NS_GO. It remains in that state for the configured green duration, changes to NS_WARN for the yellow duration, then transfers control to the East-West road. The same pattern is repeated for the East-West road.

10. MOD-N Counter

A MOD-N counter is a counter that cycles through N distinct count values, from 0 to N–1, and then returns to 0. For example, a MOD-10 counter follows 0, 1, 2, …, 9, 0. The counter can be enabled or held, and reset returns it to zero. The supplied Verilog module uses parameters so that N can be changed.

11. Hardware and Software Requirements

• Computer with a Verilog HDL simulator.

• Icarus Verilog and GTKWave, or an equivalent HDL simulation tool.

• Text editor or Verilog IDE.

• Optional FPGA development board for a classroom demonstration.

12. Inputs and Outputs

Inputs: clk (clock), reset (asynchronous active-high reset).

Outputs: ns_red, ns_yellow, ns_green, ew_red, ew_yellow, ew_green.

Only one colour is active for each road at a time. The controller is designed so that the two roads are never green simultaneously.

13. Design Methodology

• Define the required signal sequence and timing phases.

• Represent each phase as a separate FSM state.

• Create a counter that advances on each rising clock edge.

• Compare the counter with the phase duration.

• Change state and clear the counter when the phase ends.

• Decode the state into traffic-light outputs.

• Simulate the design and inspect the output sequence.

14. Verilog Implementation

The project source code is supplied as separate .v files in the project ZIP: traffic_light_controller.v, mod_n_counter.v, and tb_traffic_light_controller.v. Keeping the modules separate makes the design easier to read, test, and reuse.

15. Test Plan

Test	Action	Expected result
Reset	Set reset high	Controller starts at NS green / EW red
NS green phase	Wait configured green ticks	Changes to NS yellow / EW red
NS yellow phase	Wait configured yellow ticks	Changes to NS red / EW green
EW green phase	Wait configured green ticks	Changes to NS red / EW yellow
EW yellow phase	Wait configured yellow ticks	Returns to NS green / EW red
MOD-N counter	Count through N values	Returns to zero after N−1

16. Expected Result

During simulation, the controller should display the four phases in the specified order. At no point should both roads show green simultaneously. The counter should reset at the end of each phase. The MOD-N counter should wrap from N−1 to zero when enabled.

17. Applications

• Learning digital electronics and sequential circuits.

• Demonstrating finite-state machine design.

• Understanding clock-driven counters and timing control.

• Academic FPGA and Verilog HDL laboratory exercises.

18. Advantages

• Simple and structured control sequence.

• Predictable phase timing.

• Modular Verilog implementation.

• Easy to simulate and modify for learning.

19. Limitations

This is a simplified educational model. It does not detect real traffic, control pedestrian crossings, handle emergency vehicles, or provide redundant safety monitoring. Timing values in the example are clock-cycle counts, not real-world seconds. A real traffic installation requires professional engineering, extensive verification, fail-safe design, and compliance with relevant standards.

20. Future Enhancements

• Add vehicle sensors to estimate traffic density.

• Add pedestrian request buttons and safe crossing phases.

• Add emergency-vehicle priority under controlled conditions.

• Add configurable timing and a countdown display.

• Implement and verify the design on an FPGA board.

21. Conclusion

The project demonstrates how an FSM and counter can be combined to control a simple traffic-light sequence. Verilog HDL provides a clear way to describe the logic, while simulation helps verify state transitions and output behaviour. The design is suitable as an educational capstone starting point and can be extended with additional features.

22. References

• M. Morris Mano and Michael D. Ciletti, Digital Design, Pearson.

• Samir Palnitkar, Verilog HDL: A Guide to Digital Design and Synthesis, Prentice Hall.

• IEEE Verilog / SystemVerilog language documentation and simulator user guides.

Submission reminder: Upload the ZIP contents or repository to a Git hosting service, then submit the public project URL on the course page. Confirm that the instructor can access the files.